

Materia:
BASES DE DATOS 2 - TUIA

DATA WAREHOUSE

The Drinking Company

TRABAJO PRÁCTICO FINAL

[Github](#)

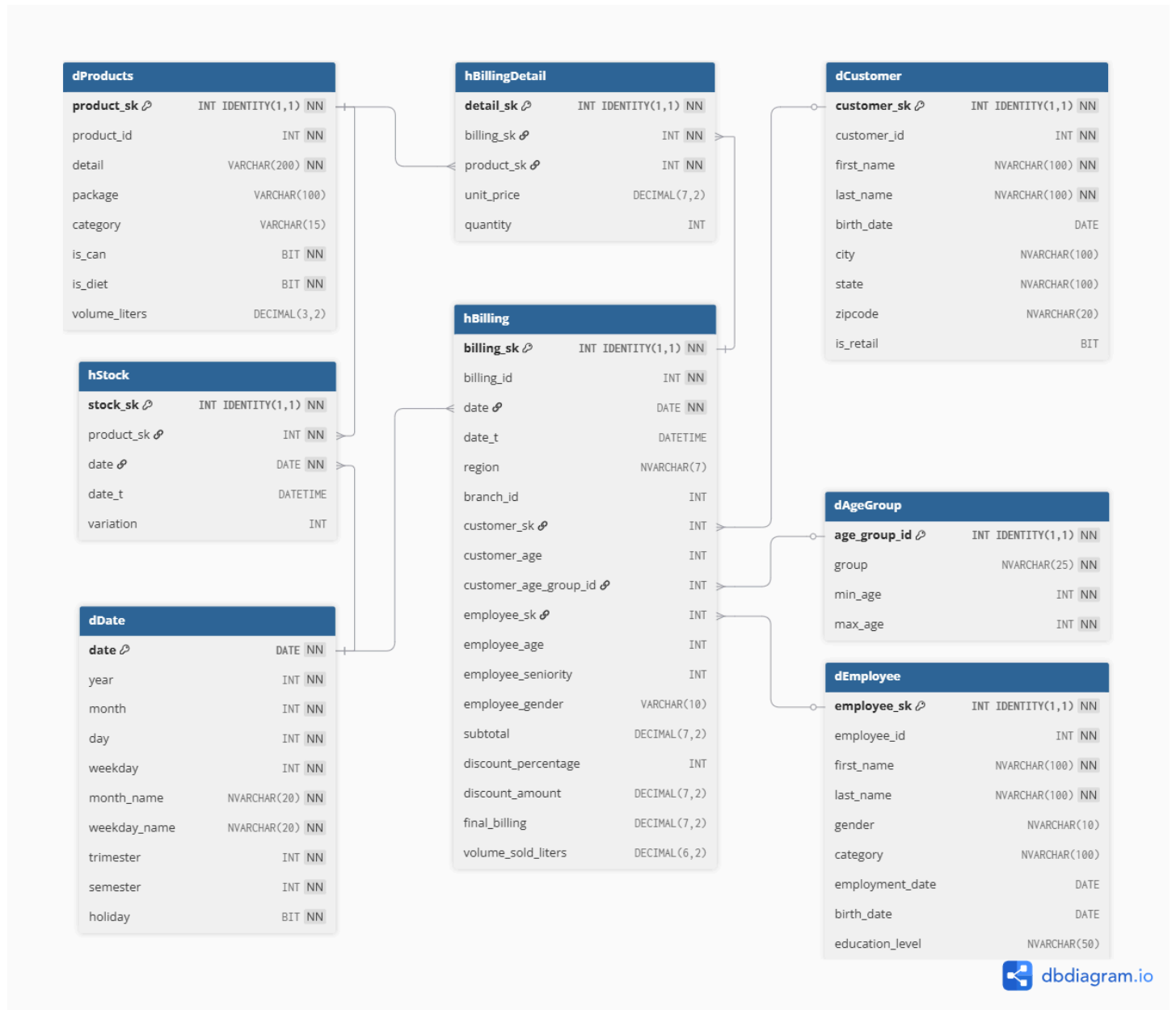
Alumnos:

Lucas Díaz Celauro
Lisandro Odisio Martinelli
Aldana Desiré Sánchez

(Grupo 20)

Modelo dimensional del Data Warehouse

El modelo diseñado sigue un esquema de tipo Estrella (Star Schema) adaptado, con una separación entre cabecera y detalle para las ventas, y múltiples tablas de dimensiones desnormalizadas para facilitar las consultas analíticas requeridas por la gerencia.



Tablas de Dimensiones

Estas tablas contienen el contexto descriptivo de las operaciones de The Drinking Company, permitiendo filtrar y agrupar la información desde distintos ángulos.

Dimensión	Descripción	Atributos Clave
dDate	Dimensión temporal que permite analizar los datos a través del tiempo. La clave es por día.	Año, mes, trimestre, semestre, día de la semana y si es feriado.
dCustomer	Centraliza a todos los clientes de la empresa.	Identificador, nombre, fecha de nacimiento, ubicación geográfica (ciudad, estado) y un indicador de si es cliente minorista (retail).
dEmployee	Almacena los datos del personal.	Género, categoría, fecha de contratación, fecha de nacimiento y nivel educativo.
dAgeGroup	Permite segmentar a las personas según rangos etarios predefinidos.	Nombre del grupo, edad mínima y edad máxima.
dProducts	Catálogo desnormalizado de todas las bebidas comercializadas.	Detalle, envase, categoría (cervezas, colas, etc.), banderas lógicas (si es lata, si es dietética) y volumen en litros.

Tablas de Hechos

Estas tablas almacenan las métricas cuantitativas del negocio y conforman el núcleo del Data Warehouse. El modelo presenta tres tablas de hechos distintas:

Tabla de Hechos	Granularidad	Descripción y Métricas
hBilling	Una fila por factura (Cabecera).	Registra la operación general de venta. Sus métricas incluyen el subtotal, porcentaje y monto de descuento, facturación final y volumen total vendido en litros. También pre-calcula la edad del cliente y del empleado y la antigüedad del empleado al momento de la venta.
hBillingDetail	Una fila por producto dentro de cada factura (Detalle).	Desglosa la venta relacionando la cabecera de la factura con cada producto específico. Sus métricas son la cantidad vendida y el precio unitario vigente.
hStock	Una fila por producto y fecha de movimiento.	Permite rastrear el inventario de la empresa a lo largo del tiempo. Su métrica principal es la variación (ingreso o egreso) del stock.

Criterios aplicados

El diseño de este modelo dimensional tipo estrella adaptado fue guiado por los requerimientos analíticos solicitados por la Dirección de The Drinking Company. A continuación, se detallan las decisiones arquitectónicas tomadas:

Separación de Facturación en Cabecera y Detalle

Se decidió dividir los hechos de ventas en dos tablas (*hBilling* y *hBillingDetail*) para manejar adecuadamente las distintas granularidades del negocio.

- **hBilling:** Permite responder consultas a nivel global de la operación (por ejemplo, analizar los descuentos totales otorgados o el rendimiento de un vendedor por factura) sin duplicar montos.

- *hBillingDetail*: Permite descender al nivel de artículo para analizar el comportamiento de productos individuales (cantidades vendidas, precios unitarios).

Desnormalización en la Dimensión Productos

En lugar de mantener un esquema altamente normalizado (tipo Copo de Nieve), se optó por desnormalizar la jerarquía de productos e incorporar atributos derivados directamente en la dimensión para optimizar el rendimiento de los tableros en Power BI:

- Banderas Lógicas (*is_diet*, *is_can*): Se crearon atributos booleanos para identificar rápidamente si una bebida es dietética o si su envase es en lata. Esto responde de manera directa a la sospecha de la gerencia sobre la pérdida de popularidad de las bebidas Diet y el consumo en lata (Consultas 13 y 14).
- Categorización Directa (*category*): Agrupar los productos por rubro facilita enormemente la segmentación para la campaña de bebidas "Energy Drink" (Consulta 5) y el análisis de stock (Consulta 16).
- Volumen Estandarizado (*volume_liters*): Para calcular la cantidad de litros consumidos (Consultas 1 y 2), se extrajo numéricamente la capacidad de cada envase en la etapa de ETL, evitando operaciones de conversión de texto durante la consulta analítica.

Cálculo de Edades y Antigüedad en hBilling

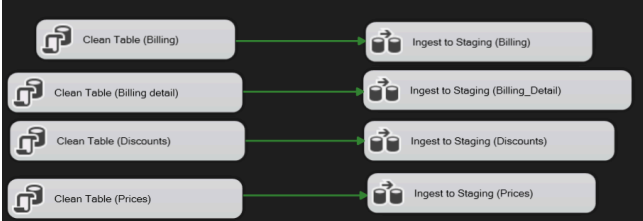
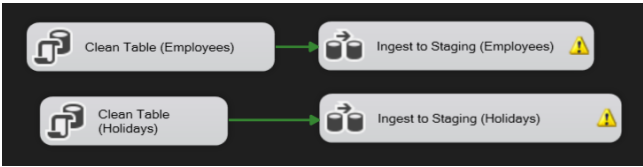
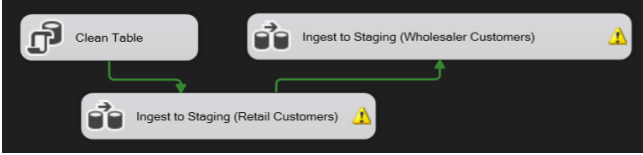
La idea usada es que las tablas de hechos deben capturar la realidad exacta en el momento en que ocurrió el evento. Por este motivo, variables como la edad o la antigüedad no se pre-calculan en el ETL y se guardan como un dato estático en la tabla de hechos:

- *customer_age* y *dAgeGroup*: Se calculó la edad del cliente al momento de la compra para poder cruzarla con el consumo de bebidas (Consulta 6). Además, se vinculó a una dimensión específica de grupos etarios para segmentar ágilmente a adolescentes, adultos medios y el rango específico de 66 años solicitado por el gerente (Consulta 7).
- *employee_age* y *employee_seniority*: Se calculó la edad y la antigüedad del empleado (diferencia entre la fecha de venta y la fecha de contratación) en el instante de cada factura. Esto es vital para que Recursos Humanos analice el rendimiento en ventas cruzado por edad, género y años de experiencia en la empresa (Consultas 8 y 10). Si calculáramos esto con la fecha actual, estaríamos distorsionando la información histórica.

Arquitectura del Proceso ETL (Control Flow)

Para poblar el Data Warehouse de The Drinking Company, se diseñó un paquete ETL en Microsoft Visual Studio organizado en un flujo de control (Control Flow) de tres grandes fases, garantizando la trazabilidad y la correcta transformación de los datos:

1. **Staging Area:** En primer lugar, se realizó la ingesta cruda de todas las fuentes de datos heterogéneas (archivos XML de clientes, TXT de producción y regiones, Excel de empleados y feriados, y las bases de datos de ventas de SQL Server y MySQL) hacia una base de datos de Staging en SQL Server. Esto se realizó en 5 paquetes de SSIS:

	SSIS_Pkg_1: Importa desde la base de datos <i>sales</i> de MySQL, usando una conexión ODBC. Primero limpia las tablas de staging, y luego carga las tablas de la base de datos <i>sales</i>
	SSIS_Pkg_2: Importa desde la base de datos <i>historySales</i> de SQL Server con una conexión OLE DB. Limpia la tabla destino primero y luego la ingesta en staging.
	SSIS_Pkg_3: Este es el paquete que importa los archivos .xlsx a las respectivas tablas de staging.
	SSIS_Pkg_4: Este paquete importa en tándem las dos fuentes de datos XML de clientes: minoristas y mayoristas. En el flujo de datos, hay una tarea de columna derivada que agrega <i>is_retail</i> como 1 si está importando la minorista, y 0 si está importando la mayorista.
	SSIS_Pkg_5: Este paquete da la ingesta de los archivos en formato plano a sus respectivas tablas del staging. Es importante en el staging considerar las columnas como string para evitar transformaciones con posibles errores que provienen de este formato.

2. **Base de Datos Intermedia:** A partir del Staging, los datos fluyen hacia una base de datos intermedia. Esta base replica la estructura dimensional final (Dimensiones y Hechos), pero

añade tablas para el manejo de errores e incorpora las tablas transaccionales de precios y descuentos. Es aquí donde se realizan los cálculos complejos y la consolidación de métricas.

3. Carga del Data Warehouse: Finalmente, una vez que los datos están limpios, consolidados y con sus claves subrogadas correspondientes, se realiza la carga final hacia el esquema estrella de TDC_DataWarehouse.

Inicialización de Dimensiones y Fechas

Previo a la ingesta de datos transaccionales, se configuró un script en la base intermedia para poblar las dimensiones con registros de inferencia (Dummy Records). Esto es vital para mantener la integridad referencial cuando un hecho no tiene una dimensión asociada. Se insertaron los ID -1 (Nulo), -2 (Inconsistente / Error) y -3 (No Registrado) en las tablas de Clientes, Empleados y Productos.

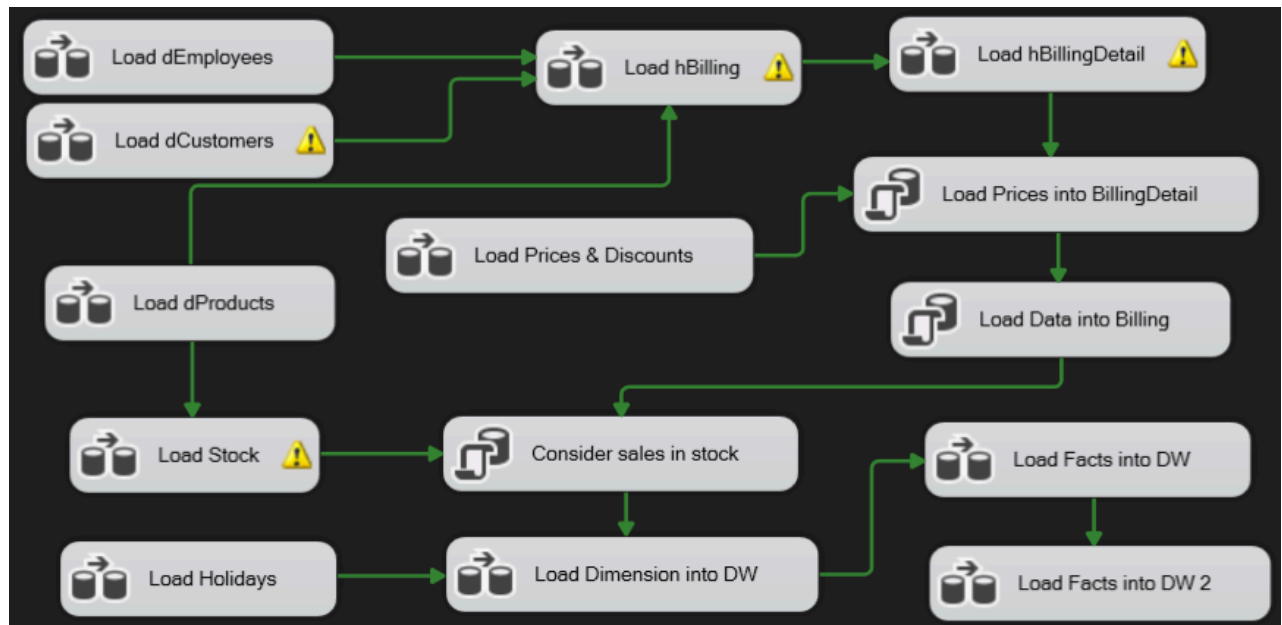
Además, se generó la dimensión temporal (*dDate*) mediante un bucle, abarcando desde el 01/01/2000 hasta el 31/12/2020, incluyendo dos fechas por defecto (1900-01-01 y 1901-01-01) para mapear fechas nulas o inconsistentes provenientes de los sistemas de origen.

Reglas de Limpieza y Transformación de Datos

Durante el flujo de datos, nos encontramos con diversas anomalías que requirieron la aplicación de transformaciones específicas en SSIS:

- Manejo de Nulos: Los valores nulos encontrados en la tabla de facturación actual (*Billing*) y en las ventas históricas (*HistorySales*) para fechas, clientes, empleados o productos, fueron redirigidos a sus correspondientes claves subrogadas de valor -1 (Nulo).
- Resolución de Granularidad (History Sales): Al no estar completamente normalizada la tabla de ventas históricas, se detectaron casos donde un mismo número de factura (*billing_id*) estaba asociado a dos empleados distintos. Estos casos se normalizaron asignando la clave de empleado -2 (Inconsistente).
- Transformación de Fechas de Nacimiento (Clientes): Se detectaron 5 registros con errores críticos en el campo *birth_date*:
 - En 3 de ellos, el error se debía a caracteres alfanuméricos inválidos, los cuales fueron limpiados mediante expresiones para permitir su conversión.
 - En los 2 restantes, el día o el mes carecían de sentido lógico. En estos casos, se extrajo únicamente el año y se imputó la fecha al primero de enero de dicho año. Adicionalmente, se verificó que no existieran fechas de nacimiento irreales (anteriores a 1930 o posteriores a 1990), confirmando la validez del resto del padrón.
- Manejo de Registros Huérfanos: En las ventas actuales, se identificaron registros en *Billing_Detail* cuyos *billing_id* no existían en la tabla de cabecera (*Billing*). Para no perder esta información de ventas, se generaron registros sintéticos en la tabla de cabecera para esos ID, dejando las claves foráneas de las dimensiones en nulo (-1).
- Verificación de Colisiones: Se comprobó la no superposición de los números de factura (*billing_id*) entre los sistemas de ventas actuales e históricos, asegurando que los componentes de búsqueda (Lookup) funcionaran correctamente al cruzar los datos.

Paquete de ETL



Para coordinar de forma automatizada y segura las múltiples etapas de Extracción, Transformación y Carga (ETL), se diseñó un Paquete Maestro (Master_Pkg) en SQL Server Integration Services (SSIS). La principal ventaja de esta arquitectura es la modularidad: permite separar los flujos de datos en paquetes independientes y controlar su ejecución desde un único tablero centralizado mediante tareas de ejecución de paquete y restricciones de precedencia.

Con la presencia de la totalidad de los datos crudos en Staging, el flujo avanza hacia la base de datos intermedia. Aquí el paquete maestro explota el paralelismo para optimizar los tiempos de procesamiento, dividiendo la carga en dos subfases críticas:

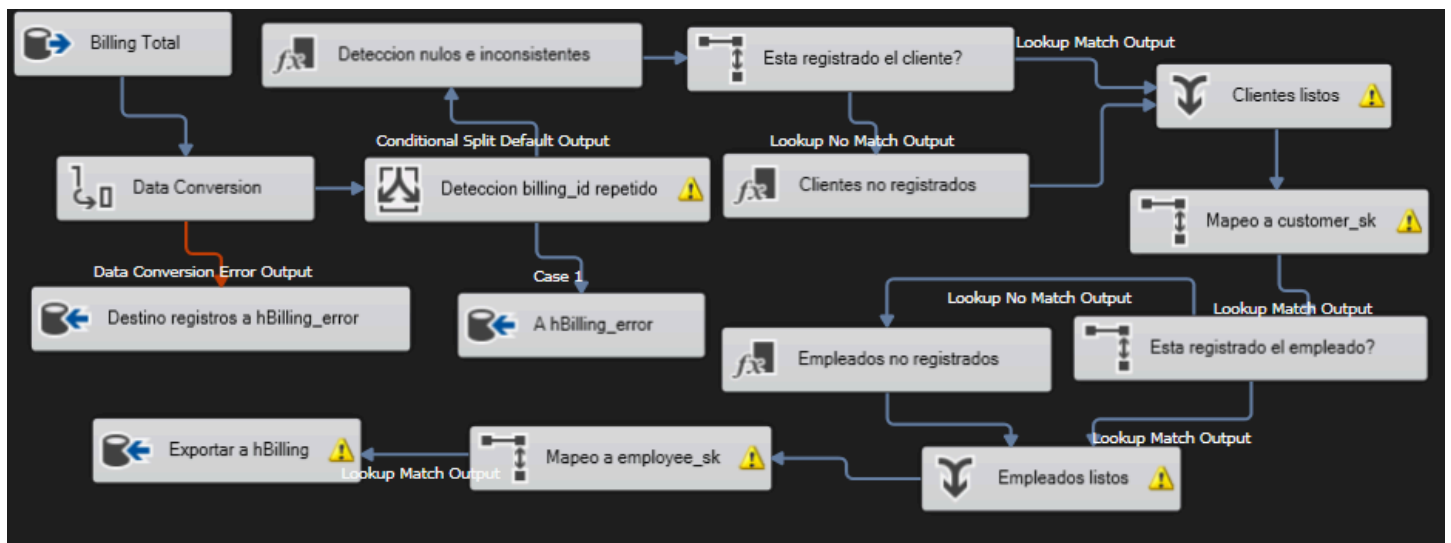
- **Procesamiento en Paralelo de Dimensiones:** Las dimensiones (*dProducts*, *dCustomer*, *dEmployee* y *dAgeGroup*) se ejecutan simultáneamente en hilos separados. Dado que son entidades independientes entre sí y sus registros de inferencia (valores dummy -1, -2, -3) ya se encuentran pre-cargados, no requieren esperar la finalización de otra tarea. La dimensión temporal *dDate* sirve de base transversal.
- **Procesamiento Secuencial de Tablas de Hechos (Control de Granularidad):** Las tablas de hechos presentan dependencias estrictas de clave foránea, por lo que su orden de ejecución está estrictamente controlado:
 1. Primero se ejecuta el paquete de *hBilling* (Cabecera), donde se unifican las ventas actuales e históricas, se resuelven los conflictos de nulos/inconsistencias. Esta tarea genera las claves subrogadas primarias de facturación.
 2. Una vez que *hBilling* finaliza con éxito, la restricción de precedencia habilita la ejecución de *hBillingDetail* (Detalle). Esto es fundamental, ya que el detalle requiere realizar búsquedas (*Lookups*) sobre la cabecera ya procesada para asociar correctamente cada producto vendido a su factura correspondiente.
 3. Luego, con las claves subrogadas calculadas, se cruzan los datos de las dimensiones y de las tablas transaccionales de precios y descuentos, para pre-calcular las edades, antigüedad, volumen total, precio del ítem, subtotal, descuento.
 4. En paralelo o de forma independiente, se procesa la tabla de hechos de inventario *hStock*, cruzando los movimientos de producción con la dimensión de productos validada. Luego de cargarla con las variaciones positivas de producción, se cargan las

variaciones negativas por ventas, proviniendo de la tabla *hBillingDetail* y *hBilling*. Es por esto que ese nodo (Consider sales in stock) debe esperar a la carga de datos de las 3 tablas.

Por último, cuando la base intermedia se encuentra completamente poblada, limpia y balanceada, se dispara la fase de carga final hacia *TDC_DataWarehouse*.

Al haberse resuelto toda la lógica pesada de negocio, limpieza de fechas e inconsistencias de claves en la etapa intermedia, el pasaje al Data Warehouse consiste en una transferencia directa de alta velocidad (*Fast Load*), asegurando que el repositorio analítico final esté disponible para Power BI sin demoras y con consistencia absoluta.

Tarea de carga de hBilling



La carga de la tabla de cabecera de facturación (*hBilling*) hacia la base de datos intermedia representa uno de los flujos de datos más complejos del proyecto. Para optimizar el rendimiento y garantizar la integridad de los datos, la estrategia se dividió en dos partes: una consulta SQL en el origen para consolidar los datos, y un flujo de transformaciones en SSIS para asignar claves y calcular métricas.

Consolidación en el Origen (OLE DB Source)

En lugar de cargar todas las tablas crudas a la memoria de SSIS y unir las allí, se diseñó una consulta SQL (ver Anexo) que realiza el trabajo pesado directamente en el motor de base de datos de Staging. Esta consulta se encarga de:

- Une mediante UNION ALL las ventas históricas (*HistorySales*) y las actuales (*Billing*).
- Identifica los detalles de facturación (*Billing_Detail*) que no poseen una cabecera y les genera un registro temporal ("Orphan") con IDs nulos para no perder la venta.
- Agrupa por *billing_id* y cuenta cuántos empleados distintos están asociados a una misma factura (*emp_count*), preparando el terreno para resolver problemas de granularidad.
- Asegura que solo ingrese una fila por cada número de factura al flujo de SSIS.

Transformaciones en SSIS (Data Flow)

Una vez que el motor SQL entrega el set de datos consolidado, el flujo de Integration Services (ilustrado en la figura) aplica las siguientes transformaciones secuenciales:

- **Derived Column (Nulos a -1):** Se interceptan los valores nulos provenientes de las fechas, clientes o empleados huérfanos y se los reemplaza por el valor -1. Esto asegura que más adelante el cruce con las dimensiones asigne el registro "Nulo" predefinido, manteniendo la integridad referencial.
- **Derived Column (Reemplazo Emp por -2):** Utilizando la bandera emp_count generada en el origen SQL, se evalúa si una factura tiene múltiples vendedores asignados (error de granularidad de las ventas históricas). Si la cuenta es mayor a 1, se fuerza el employee_id a -2 (Inconsistente).
- **Lookups de Dimensiones (dCustomer y dEmployee):** El flujo cruza los IDs del sistema operacional con las dimensiones de la base intermedia para obtener las Claves Subrogadas (customer_sk y employee_sk). Se usan también para, en conjunto con una Derived Column, para encontrar los clientes y empleados que no están registrados.
- **Carga Final (OLE DB Destination):** Finalmente, los registros completamente transformados, con sus claves subrogadas y métricas calculadas, se insertan mediante en la tabla *hBilling* de la base de datos Intermedia.

Anexo: SQL query para importar los datos consolidados para hBilling

```

WITH HistoryBase AS (
    SELECT DISTINCT
        billing_id,
        date,
        date AS date_t,
        customer_id,
        employee_id,
        CAST(NULL AS VARCHAR(7)) AS region,
        CAST(NULL AS VARCHAR(10)) AS branch_id,
        'History' AS source_table
    FROM HistorySales
),
SalesBase AS (
    SELECT DISTINCT
        billing_id,
        date,
        date AS date_t,
        customer_id,
        employee_id,
        region,
        branch_id,
        'Sales' AS source_table
    FROM Billing
),
OrphanDetails AS (
    SELECT DISTINCT
        bd.billing_id,
        CAST(NULL AS VARCHAR(100)) AS date,
        CAST(NULL AS VARCHAR(100)) AS date_t,
        CAST(NULL AS VARCHAR(100)) AS customer_id,
        CAST(NULL AS VARCHAR(100)) AS employee_id,
        CAST(NULL AS VARCHAR(7)) AS region,
        CAST(NULL AS VARCHAR(10)) AS branch_id,
        'Orphan' AS source_table

```

```

FROM Billing_Detail bd
WHERE NOT EXISTS (SELECT 1 FROM Billing b WHERE b.billing_id = bd.billing_id)
AND NOT EXISTS (SELECT 1 FROM HistorySales hs WHERE hs.billing_id = bd.billing_id)
),
Combined AS (
    SELECT * FROM HistoryBase
    UNION ALL
    SELECT * FROM SalesBase
    UNION ALL
    SELECT * FROM OrphanDetails -- Unimos los huérfanos al flujo general
),
AggregatedFlags AS (
    SELECT
        billing_id,
        COUNT(DISTINCT source_table) AS source_count,
        COUNT(DISTINCT employee_id) AS emp_count,
        COUNT(DISTINCT customer_id) AS cust_count
    FROM Combined
    GROUP BY billing_id
),
DeduplicatedData AS (
    SELECT
        *,
        ROW_NUMBER() OVER(PARTITION BY billing_id ORDER BY source_table) AS rn
    FROM Combined
)
SELECT
    d.billing_id,
    d.date,
    d.date_t,
    d.customer_id,
    d.employee_id,
    d.region,
    d.branch_id,
    d.source_table,
    f.source_count,
    f.emp_count,
    f.cust_count
FROM DeduplicatedData d
INNER JOIN AggregatedFlags f ON d.billing_id = f.billing_id
WHERE d.rn = 1;

```

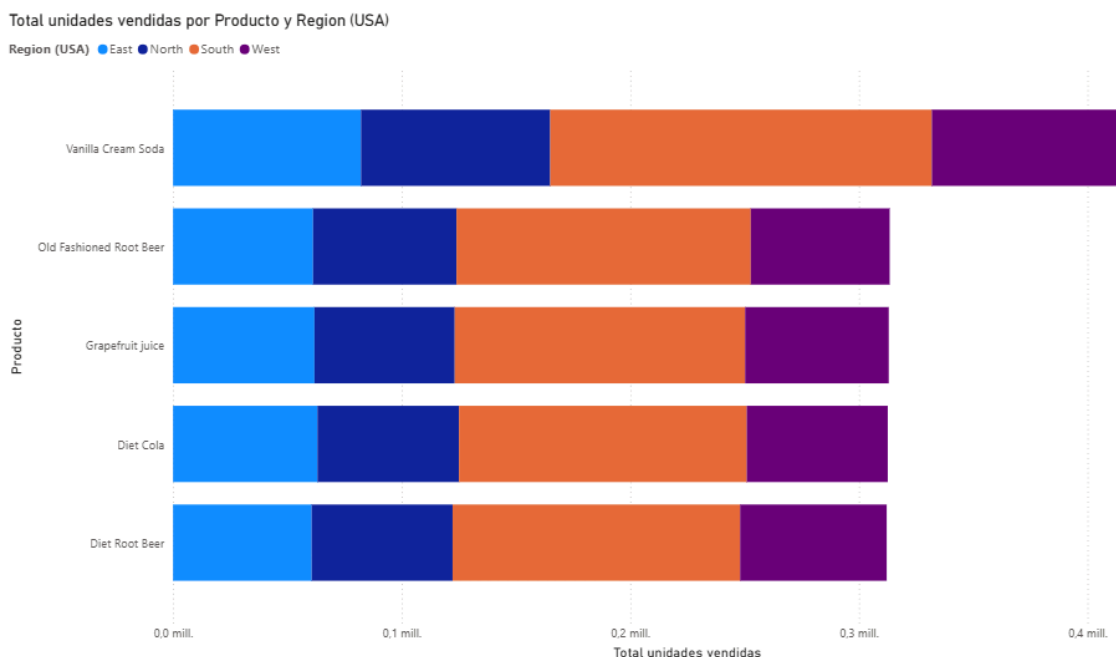
Ejemplo de uno de los reportes diseñados en Power BI

Se diseñó un tablero en Power BI con el objetivo de analizar el comportamiento de ventas de la empresa en función de los productos, las zonas geográficas y su evolución en el tiempo. El análisis se basa en un Data Warehouse que integra información de ventas, productos, clientes y fechas.

Para construir el modelo se utilizó la tabla de hechos “hbillingDetail”, donde se definió la medida “Monto de Ventas” como el producto entre el precio unitario y la cantidad vendida, permitiendo cuantificar el total de ventas por transacción. A partir de esta medida, se construyeron visualizaciones que permiten segmentar la información por producto, región geográfica y año.

El tablero cuenta con dos visualizaciones principales:

1. Un gráfico de barras para el Top 5 de productos más vendidos, donde se suman sus ventas por zona geográfica (diferenciadas por color). Desde Power BI, este gráfico se puede visualizar para un año en particular o para el historial completo. Al exportarlo como PDF perdemos esta opción, y sólo podemos quedarnos con un año en particular (o todo el historial).



2. Una tabla de detalle que muestra el comportamiento del TOP 5 de ventas por producto, región y año. Se ve que los registros con información de región (state) inician y terminan en año 2009 ya que fue el único año donde se solicitó llenar este campo en la BDD. Por eso, el análisis por zona geográfica queda entendido como un corte en ese año, y sirve para comparar ventas solo del 2009.

Producto	Region (USA)	Total unidades vendidas
Diet Cola	East	63163
Diet Root Beer	East	60632
Grapefruit juice	East	61852
Old Fashioned Root Beer	East	61239
Vanilla Cream Soda	East	82229
Diet Cola	North	61900
Diet Root Beer	North	61640
Grapefruit juice	North	61365
Old Fashioned Root Beer	North	62740
Vanilla Cream Soda	North	82754
Diet Cola	South	125884
Diet Root Beer	South	125773
Grapefruit juice	South	127004
Old Fashioned Root Beer	South	128699
Vanilla Cream Soda	South	166913
Diet Cola	West	61663
Diet Root Beer	West	64086
Grapefruit juice	West	62812
Old Fashioned Root Beer	West	60766
Vanilla Cream Soda	West	84417
Total		1667531

El gráfico de barras apiladas, a diferencia de la tabla con la misma información, permite visualizar de forma más clara la distribución geográfica de las ventas para cada producto. Vanilla Cream Soda se destaca como el producto con mayor volumen total de unidades vendidas, superando las 0,4 millones de unidades acumuladas entre todas las regiones, lo que lo posiciona como el líder indiscutido del portafolio de TDC.

En cuanto a la distribución regional, la región South representa consistentemente el segmento más amplio dentro de cada barra, evidenciando que es la zona geográfica con mayor demanda para todos los productos sin excepción. Las regiones East y North muestran contribuciones similares y más moderadas, mientras que West aporta el menor volumen relativo en la mayoría de los casos.

El resto de los productos —Old Fashioned Root Beer, Grapefruit Juice, Diet Cola y Diet Root Beer— presentan volúmenes totales similares entre sí, rondando los 0,3 millones de unidades, con una distribución regional prácticamente homogénea, lo que sugiere que la preferencia por Vanilla Cream Soda es la única diferenciación significativa dentro del catálogo.

En síntesis, la combinación del liderazgo de Vanilla Cream Soda con la dominancia de la región South representa el segmento de mayor relevancia comercial para la compañía, y debería ser considerado como eje central en las decisiones de abastecimiento y estrategia de ventas.